Creating a Pipeline to scRNA-seq Data in Go

Vania Halim, Qinglin Kong, Amy Ji, Yinan Elise Zhu

Introduction

A central goal in modern biology is to understand how genotypes give rise to phenotype. In decades of research, transcriptomic profiling has been one of the most powerful tools for mapping this relationship. By quantifying gene expression, we can approximate the functional state of cells, tissues, and even organisms. Traditional approaches such as RNA sequencing (RNA-seq) have pushed the field forward by allowing us to understand many disease mechanisms, developmental pathways, and cellular systems. However, in many cases, when a tissue gets digested in a test tube, gene expression signals are averaged across thousands or millions of cells, masking heterogeneity and hiding the contributions of rare or transient cell populations. In cancer, immunology, or developmental biology, this heterogeneity is often not noise, but a defining feature of a phenotype that could be very important.

The limitation of bulk RNA-seq led to the development of single-cell (sc)RNA-seq, which profiles the transcriptome down to individual cell level, rather than pooled populations. By looking at these biological processes at a single-cell resolution, scRNA-seq enables researchers to identify discrete cell states, reconstruct differentiation trajectories, detect tumor subclones, and quantify immune cell diversity. Recent work, such as one published in *Cancer Cell* mapping neutrophil states across tumor progression (Lad et al., 2024), shows how single-cell transcriptomics uncovers distinct, functional neutral programs that bulk measurements would mask any day. These findings highlight the power of single-cell approaches for linking gene expression variation to phenotypic behavior in complex tissues, especially in this case, solid tumors like glioblastoma.

However, it is important to note that the shift from bulk to single-cell RNA-seq has changed the computational problem for data analysis. Instead of summarizing a single averaged gene expression vector, we now have datasets of many cells, each represented as high-dimensional and noisy vectors. The biological question of how genes determine cellular identity is now a much more complicated computational problem of dimensionality reduction, neighborhood graph construction, and community detection. Modern single-cell analysis tools such as Seurat and Scanpy implement these steps using algorithms like Principal Component Analysis (PCA), K-nearest neighbor (KNN) graphs, and the Leiden clustering algorithm. In this project, we aim to reimplement these methods to understand how they transform raw single-cell gene expression data into interpretable biological

structure. The ultimate goal of this project is to create a tool where researchers can select the appropriate method and customize the pipeline for their unique datasets.

Methods

Preprocessing: Quality Control

Before any computational modeling can happen, scRNA-seq data requires substantial preprocessing. Raw single-cell count matrices are extremely sparse, zero-inflated, and subject to high amounts of technical variation. This noise comes primarily from the droplet-based microfluidics described by Macosko et al. (2015) and used in major platforms like the 10X Genomics Chromium, where beads and cells are randomly encapsulated together into a tiny oil droplet. Beads are tiny resin particles that have millions of DNA fragments covalently attached on its surface. Each bead carries a cell barcode identifying the cell, and UMI barcodes identifying individual transcripts from that cell.

Ideally, each droplet should contain exactly one bead and one intact cell, but in practice, many issues can occur during microfluidic encapsulation. Cells undergoing apoptosis or necrosis can release their DNA into the environment, and because nucleic acids are inherently sticky, single cells may clump together. When two cells are trapped in the same droplet, the resulting barcode shows abnormally high total RNA counts and inflated gene diversity. This is what we call doublets.

Conversely, droplets can also capture RNA from ruptured cells without an intact cell present, producing barcodes with very low total RNA content. Stressed or dying cells also have elevated mitochondrial gene expression. Because mitochondria have a double membrane and maintain structural integrity longer than the rest of the cell, mitochondrial RNA is often preserved and captured even when the cytoplasm has been lost. As a result, droplets containing dying or damaged cells exhibit unusually high percentages of mitochondrial transcripts.

These types of artifacts distort downstream analysis. Without proper quality control, PCA may be dominated by mitochondrial RNA rather than biological structure, KNN graphs may link unrelated artifacts, and clustering algorithms like Leiden may produce communities based on technical artifacts rather than true biological phenotypes.

In many analysis pipelines, similar metrics are being used to identify low-quality barcodes. For Seurat in particular, the following metrics are called *nFeature_RNA*, *nCount_RNA*, and *percent.mt*. *nFeature_RNA* counts the number of unique features, or genes, found in a particular cell; *nCount_RNA* counts the total number of UMIs, or individual mRNA transcripts, detected in each cell; *percent.mt* calculates the percentage of reads that map to mitochondrial genes.

Computational Problem

Given a raw count matrix *CountMatrix* where each row represents a cell, we determine which rows correspond to real, biologically meaningful cells and which are technical artifacts. The computational objective is to identify and retain only the indices of cells whose QC metrics fall within acceptable biological ranges, removing barcodes representing doublets, debris, ruptured cells, or low-quality transcriptomes.

High-level algorithmic solution

The algorithmic solution for QC is simplistic. A *CountMatrix* method *FilterBy()* takes in as input a set of user-defined *nFeature_RNA*, *nCount_RNA*, and *percent.mt* thresholds and iterates over each barcode in the count matrix to evaluate whether the cell satisfies all constraints. Once this set of valid indices is assembled, the filtered count matrix is constructed by subsetting the original data to retain only those cells.

Preprocessing: Normalization

After quality control, the next step in the pipeline is to normalize the counts. Raw UMI counts are heavily influenced by technical variability rather than biology. Even between high-quality barcodes, some cells may have more total RNA than others due to differences in cell sizes, transcriptional activity, or sequencing depth. These differences may mask the true biological signals needed. For example, two cells might both express the same genes, but if one cell has twice as many total counts as the other, the gene expression for that cell will appear artificially stronger.

Computational Problem

The computational problem for normalization is to transform a CountMatrix into a new matrix where differences reflect underlying biology rather than technical variation. Modern workflows tend to have multiple types of normalization methods, the most widely used being log-normalization or some variations of a model-based residual method.

With log-normalization, each cell is first scaled to a common total, for example, counts per million, which removes global depth effects. Then, a log-transform is applied to stabilize variance and compress the wide dynamic range of scRNA-seq expression values. This approach is fast and simple, and is the default method used in many tutorials.

Newer versions of Seurat use a model-based residual method called Pearson residual normalization. Instead of just scaling total counts, Pearson residuals estimate the expected expression of each gene in each cell using a depth-only model, then measure how far the observed counts deviate from this expectation. It is a more direct capture of biological structure while reducing technical noise.

In our project, we implemented both methods so that we can compare a classical approach with a more rigorous model-based alternative.

High-Level Algorithmic Solution

We first construct an expression matrix, *ExpressionMatrix*, from the filtered *CountMatrix* by placing cells in rows and genes in columns using the *BuildMatrix()* function. This new *ExpressionMatrix* is the basis of both types of normalization strategies.

We implement classical log-normalization exactly as originally described in Seurat, where each cell is scaled to a fixed total count and the values are transformed. The exact algorithm for log normalization is as follows:

$$X_{cg}^{\log} = \log\left(1 + \frac{X_{cg}}{\sum_{g'} X_{cg'}} \cdot s\right) \tag{1}$$

Let X be the raw count matrix, where X_{cg} denotes the total number of UMI counts for gene g in the cell c. For each cell c, $\sum_{g'} X_{cg'}$ is the total UMI count in that cell. The constant s is a user-specified scaling factor, which sets the target total counts after normalization. Normalization is applied entry wise in the matrix. We first divide the raw count by the total UMI count, which we get a fraction of the cell's total expression. We then multiply by s, so that all cells are rescaled to have approximately the same total counts. Finally, we take the log of one plus this scaled value, which compresses the dynamic range and stabilizes the variance. The result is the log-normalized value $X_{cg}^{\log}$. We define an *ExpressionMatrix* method *Normalize(s)*, which runs both calculations by calling methods *NormalizeCPM(s)* and *Log1p()*.

At this point, each gene may exhibit a different mean and variance, driven partly by technical factors such as sequencing depth and partly by intrinsic biological variability. Seurat applies gene-wise standardization, transforming each gene to have mean zero and unit variance. This step is called scaling, and it's being done to prevent highly expressed genes from dominating the principal components during dimensional reduction using PCA. Mathematically, for each gene g, we compute the mean μ_g and standard deviation σ_g across all cells, transforming each value using z-scoring:

$$Z_{cg} = \frac{X_{cg}^{log} - \mu_g}{\sigma_g} \tag{2}$$

We also implement Pearson residuals, which is another commonly used method that reframes normalization as a variance-stabilization problem. First, instead of simply scaling each cell to a fixed

total like we did in log normalization, the idea is to model what expression should look like if only technical sequencing depth varied across cells. First, we estimate an expected *ExpressionMatrix* μ_{cg} using a depth-only model, where the expected abundance of a gene in a cell is proportional to the cell's total UMI count and the gene's overall abundance across cells (Lause et al., 2021). Our *Expected()* method produces the following:

$$\mu_{cg} = \frac{\sum_{g'} X_{cg'} \cdot \sum_{c'} X_{c'g}}{\sum_{c'g'} X_{c'g'}} \tag{3}$$

Then, the normalized value Z_{cg} is computed by an *ExpressionMatrix* method *Pearson(θ)*:

$$Z_{cg} = \frac{X_{cg} - \mu_{cg}}{\sqrt{\mu_{cg} + \mu_{cg}^2/\theta}} \tag{4}$$

Here, the square root of $\mu_{cg} + \mu_{cg}^2/\theta$ is the variance of a negative binomial model with mean μ_{cg} and overdispersion θ. The overdispersion parameter requires user input. In practice, common pipelines use $\theta \approx 100$. The result Z_{cg} measures how much the observed count deviates from what is expected under a negative binomial model.

Both methods produce a Z_{cg} value where positive numbers mean over-expression and negative numbers mean under-expression. However, log normalization and scale data compare each gene to its mean expression across all cells, and the denominator simply standardizes variance so all genes contribute equally. In contrast, Pearson residuals compare each count to its statistically expected value under a depth-aware model, and the denominator accounts for the natural mean–variance relationship of UMI data, making the residuals more grounded in the underlying noise structure.

Finally, to prepare for PCA, we need to limit extreme outlier values from weakly expressed genes or rare cell populations. We implemented an *ExpressionMatrix* method *clipValue(z, clip)* following Lause et al., which clips Z_{cg} to $[-\sqrt{n_{\text{cells}}}, \sqrt{n_{\text{cells}}}]$.

Dimension Reduction using PCA

The key idea behind PCA in scRNA-seq is to reduce the extremely high dimensionality of gene expression data while preserving the major structure of biological variation. Each cell is represented by thousands of gene expression values, and many of these genes are correlated because they participate in shared biological spaces and programs. PCA extracts the dominant patterns of co-expression by identifying the directions in gene space along which cells vary the most.

Mathematically speaking, these dominant directions are captured by eigenvectors of the covariance matrix. Biologically speaking, each eigenvector is essentially a type of gene expression program, where some genes' expression increase and others decrease together. The associated eigenvalues quantify how much variance that program explains across all cells. Then, by projecting the dataset onto a small number of these principal components (PCs), we obtain a low-dimensional representation that captures the major sources of heterogeneity while suppressing noise. This information is then used for graph-based clustering.

Computational Problem

The computational problem is to transform this high-dimensional matrix into a smaller set of orthogonal eigenvectors such that each eigenvector captures a maximal amount of variance in the data and explains variance that is independent of previous components. Then, we can project cells onto these eigenvectors to obtain low-dimensional embeddings for clustering and visualization.

Formally, given a centered expression matrix $\tilde{X}_{cg} \in \mathbb{R}^{n \times d}$, where *n* is the number of cells and *d* is the number of genes, the problem is equivalent to finding the top-*k* eigenvector. In our project, we delegate eigendecomposition to Gonum's *stat.PC* routine, which computes principal components for us. However, we still have to project the cell matrix onto those directions by ourselves.

High-Level Algorithmic Solution

To perform PCA, we first convert the normalized *ExpressionMatrix* into a contiguous dense matrix X_{cg} using our *ToDense()* method. This method produces a $n \times d$ **mat.Dense* object which is a matrix stored in a single 1D slice, where *n* and *d* still represent as rows/cells or columns/genes, respectively. Each **mat.Dense* objects contain fields *n* and *d* as integers which helps us convert them back to a regular 2D slice. Because Gonum provides a set of useful implementations of matrix structures as well as many linear algebra operations, we can then define *pc* as a *stats.PC* object, which is a type in Gonum for computing and extracting the principal components of a matrix.

We then get a centered matrix $\tilde{X}_{cg} \in \mathbb{R}^{n \times d}$ by subtracting each gene *g* with the mean expression μ_g.

$$\tilde{X}_{cg} = X_{cg} - \mu_g \tag{5}$$

While in normalization we produced approximately zero-centered values for X_{cg}, PCA requires exact centering so that singular value decomposition (SVD) corresponds to the eigendecomposition of the covariance matrix. Otherwise, PCA would incorrectly attribute some variance to non-zero means rather than true biological values. One thing to note is that Gonum does not use the textbook method to find eigenvectors and eigenvalues of covariance matrix *C*, which has a shape of $d \times d$.

$$C = \frac{1}{n-1}\tilde{X}^\top \tilde{X} \tag{6}$$

In the code, covariance matrix *C* is never actually constructed, because working with a full $d \times d$ matrix costs up to hundreds of millions of floats. Also, the multiplication of $X^\top X$ is extremely inefficient. Instead, our implementation performs PCA by applying a thin singular value decomposition (SVD) directly to the centered expression matrix $\tilde{X}_{cg} \in \mathbb{R}^{n \times d}$ as follows:

$$\tilde{X_{cg}} = U\Sigma V^T, \text{where } U \in \mathbb{R}^{n \times n}, \Sigma \in \mathbb{R}^{n \times d}, V \in \mathbb{R}^{d \times d} \tag{7}$$

Here, each column v_i of *V* is equivalent to an eigenvector of our centered expression matrix. v_i tells us the gene weight pattern for PC_i. The scores $XV = U\Sigma$ are the projection of each cell onto the principal components. Then, we compute the PCA scores by projecting each cell onto these components, giving its coordinates along the major axes of variation. We store our PCA results as a *PCAResult* object, where we keep the gene weights for each PC as loadings, the cell embeddings as scores. We also store the variance explained by each PC, and a total variance for computing the proportion of variance explained.

While the SVD is a sophisticated method to find principal components, its details are beyond the scope of the courses that we are taking. We rely on Gonum's well-tested implementation as a reliable computational method for obtaining principal components without forming the covariances matrix directly to save computational resources and reduce computational burdens.

t-SNE and UMAP

Principal Component Analysis reduced the dimension of data to around 30-50 principal components. Yet, such data is still too complex to be manually interpreted and visualized. Therefore, we need methods that can transform data from a high-dimensional space into a low-dimensional space while retaining the data's structure and relationships. We designed our pipeline so that users can select between two types of embedding methods: t-distributed stochastic neighbor embedding (t-SNE) and uniform manifold approximation and projection (UMAP).

High-level algorithmic solution

Unlike PCA, both UMAP and t-SNE are non-linear methods, which means that we cannot find a single matrix or a linear function to map every high-dimensional data point to its low-dimensional embedding in a consistent way. Instead, UMAP and t-SNE bend and wrap space differently in different regions based on neighborhood and similarity relationships.

1) t-SNE

Proposed by Laurens van der Maaten and Hinton, the t-SNE algorithm is an optimized variant of stochastic neighbor embedding (SNE). t-SNE uses a Student-t distribution to compute the similarity of two points in the low-dimensional space. The pairwise similarities in high-dimensional space p_{ji} is defined as:

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)} \tag{8}$$

This is a Gaussian kernel turned into a probability distribution over all neighbors, j, on a point of interest i. The numerator computes the exponentials of the squared Euclidean distance between points i and j over the squared two-tailed standard deviation. It increases as j is close to i, and it decreases when j and i are far apart. The denominator calculates and sums that value over all points. In short, t-SNE is interested in seeing how likely point j is to be picked as a neighbor of point i based on the high-dimensional probability distribution.

t-SNE also takes into account that the conditional probability p_{ji} is asymmetric, meaning that the probability of j being selected as a neighbor of i does not necessarily equal the probability of i being selected as a neighbor of j. To address this, t-SNE defines a symmetric joint probability:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n} \tag{9}$$

So that pair (i, j) receives high probability only if both i thinks j is a likely neighbor and j thinks i is a likely neighbor. This symmetric treatment of pairwise similarities distinguishes t-SNE from its predecessor, SNE, which operated directly on the asymmetric conditional probabilities and optimized an asymmetric cost function.

For the low-dimensional counterparts y_i and y_j, t-SNE defines an analogous set of joint probabilities q_{ij}, but now using a Student-t distribution with one degree of freedom:

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}} \tag{10}$$

The goal of the algorithm is then to place the $y_i's$ so that this low-dimensional similarity distribution $Q = \{q_{ij}\}$ matches the high-dimensional similarity distribution $P = \{p_{ij}\}$as closely as possible, by minimizing the Kullback–Leibler divergence: $KL(P||Q)$. In this way, the asymmetric local Gaussian neighborhoods are first turned into a symmetric, globally normalized notion of similarity, and the embedding is explicitly optimized so that the Student-t based similarities in the map faithfully reflect those high-dimensional relationships.

Ultimately, the goal of t-SNE is to search for the best possible configuration, such that points in 2D or 3D whose Student-t-based similarity q_{ij} reproduce high-dimensional Gaussian based similarity p_{ij}, thereby preserving local neighborhoods and revealing cluster structures in the data.

2) UMAP

We implemented and built UMAP from scratch, referencing the method published by McInnes et al. While the overall goal of UMAP is similar to that of t-SNE, UMAP uses a completely different approach to determine "similarity" and which points to select as neighbors.

For each point of interest x_i, the similarity score, or weight, between i and all j's are computed as follow:

$$\mu_{ij} = \exp\left(-\frac{\max\big(0,\ d(x_i, x_j) - \rho_i\big)}{\sigma_i}\right). \tag{11}$$

Where $d(x_i, x_j)$ is often the Euclidean distance of point i and j in high dimensional space. ρ_i is the distance from i to its nearest non-identical neighbor. σ_i is a local normalization parameter that needs to be adjusted so that the sum of weight of all "effective" neighbors is about $log_2(k)$, k being the number of nearest neighbors selected. We first built KNN graphs based on high dimensional Euclidean distance through the function *buildKNNForUMAP*. Let each point be a node in a KNN graph, μ_{ij} is the weight of the edge connecting *i* to all *j*'s. ρ_i and σ_i is computed for each cell using the function *computedRhoSigma*. If a point *j* has no obvious connection to i, the edge will have a weight that's approximately 0. And weight will be closer to 1 if the connection is stronger. Such directed weight relationship between nodes is stored in a new graph by the function *buildDirectedProbs*.

Similar to t-SNE, UMAP also symmetrizes these directed weight, but using a fuzzy union:

$$\mu_{ij}^{(\mathrm{sym})} = \mu_{ij} + \mu_{ji} - \mu_{ij}\,\mu_{ji} \tag{12}$$

which means that as long as one of the two directed edges exists, the two nodes will be considered connected. Using this approach, we were able to convert our original directed weight graph into an undirected fuzzy graph through the function buildFuzzyGraph.

For the low-dimensional counterparts y_i and y_j, we initialize small random embedding and run stochastic gradient descent. At each epoch we sample edges according to their fuzzy weight (heavier edges are visited more often), using parameter *epochsPerSample* to control how frequently each edge is updated. For a sampled edge(i,j), we then compute their euclidean distance and apply an attractive gradient base on the UMAP kernel:

$$\Phi(r^2) = \frac{1}{\left(1 + a\,(r^2)^b\right)^b} = \frac{1}{\left(1 + a\,\|y_i - y_j\|^{2b}\right)^b} \tag{13}$$

This pulls connected points closer together. For each positive update, we randomly select n negative point k, and apply a repulsive gradient between i and k. We repeat this for many epochs with a decaying learning rate, and finally mean-center and rescale the embedding so the overall layout has a nice plot scale. Intuitively, the optimizer is a UMAP-flavored force-directed layout: fuzzy edges pull neighbors together, and negative samples push non-neighborhoods apart.

Compared to the referenced paper our method is simplified. The paper defines an explicit fuzzy-set cross-entropy objective between the high-dimensional fuzzy graph and a low-dimensional fuzzy graph induced by the embedding, and derives gradients from that loss. We instead use closed-form "force" formulas inspired by those gradients, without explicitly recomputing low-dimensional membership probabilities or the full cross-entropy at each step. We also use a simple heuristic for choosing the kernel parameters a,b from *minDist*, and we initialize the embedding with small random noise rather than the spectral graph initialization (Laplacian eigenvectors) described in the paper. So we keep the same big ideas—fuzzy graph, UMAP kernel, weighted edge sampling, and negative sampling—but we optimize a slightly looser, force-based approximation instead of the exact probabilistic objective in the paper.

Drawbacks of using UMAP and t-SNE for visualization.

There are a couple of major drawbacks for both t-SNE and UMAP. Firstly, both methods are explicitly designed to preserve local neighborhood relationships, not long-range geometry. The distance between clusters in low-dimensional visualization can be deceiving, as they are not necessarily very far in original space.

Second, due to the randomness introduced by the likelihood models. Neither UMAP nor t-SNE can provide confidence intervals, p-values, or standard errors on cluster boundaries or positions. Therefore, we should exercise caution when exploring hypotheses using these embedding plots and refrain from treating the 2D clusters as conclusive evidence. In single-cell genomics, for example, one should not assign cell types purely by eyeballing the clusters without sufficient quantitative follow-up. Therefore, it is necessary to better understand the community relationships between and within clusters.

Third, t-SNE and UMAP are highly sensitive to hyperparameters. For t-SNE, it depends heavily on perplexity and learning rate. For UMAP, tuning *min_dist* and *n_neighbors* can control how tight clusters are and how much global structure we can observe. Some scientists even stated that users must not choose parameter settings to find the well-clustered, aesthetic plot, and thus be guilty of confirmation bias. It is important to keep in mind that parameters need to be selected in justified and appropriate ways.

Graph-based Clustering: KNN graphs

In preparation for the Leiden clustering algorithm, we construct an undirected KNN graph where the edge weights are inversely proportional to the gene expression Euclidean distance between two cells, represented as nodes.

Computational Problem

The computation problem is that, given a matrix of cell embeddings where n is the number of cells and p is the number of principal components, we want to construct a sparse weighted graph with one node per cell, such that each node i is connected to its k nearest neighbors under a chosen distance metric. For every cell i, we want to first compute its distance to all other cells j, then select the indices of the k smallest distances. And eventually add undirected edges between i and those neighbors, with weights w_{ij}, reflecting similarity.

We also want the KNN graph to be approximately symmetric: if cell j is a nearest neighbor of cell i, we would like an edge in the opposite direction as well. Biologically, this reflects the intuition that if cell 1 is closely related to cell 2, then cell 2 should also be considered closely related to cell 1. However, the raw KNN construction is inherently asymmetric—cell 2 may fall among the k nearest neighbors of cell 1, while cell 1 is not among the k nearest neighbors of cell 2 if there are other cells even closer to cell 2. To make the graph better match this biological symmetry while remaining sparse enough for efficient clustering on thousands of cells, we explicitly symmetrize the graph: for each pair of connected cells, we keep the maximum of the two directional weights and assign that value as the edge weight in both directions.

High-Level Algorithmic Solution

DistanceMatrix() takes the low-dimensional cell embeddings and computes dense pairwise distance matrix $D \in \mathbb{R}^{n \times n}$ where D_{ij} stores the distance between cell *i* and cell *j*. We iterate over all pairs of rows in the embedding matrix and call *Euclidean()* to compute the distance for each pair, then write the result into the corresponding entry of D.

Euclidean() is the core distance routine used by *DistanceMatrix()*. Given two cell vectors in the low-dimensional space, we compute their Euclidean distance by summing the squared coordinate differences and taking the square root. This scalar distance is then inserted into the appropriate entry of the distance matrix.

BuildKNNGraph() is our main entry point for constructing a KNN graph from a precomputed distance matrix. We first handle edge cases (e.g., an empty distance matrix or an invalid k). We then call *fillDirectedKNNWeights()* to compute a directed KNN weight matrix. After that, we call *symmetrizeWeightsUsing()* to convert these directed weights into an undirected adjacency map while accumulating the total edge weight. The final output is a Graph object with Nodes, Edges, and TotalWeight filled in, ready to be used in downstream Leiden clustering.

fillDirectedKNNWeights() converts the distance matrix into a directed KNN weight matrix. For each cell r, we extract the vector of distances from r to all other cells, call getNeighborsRow() to construct a list of candidate neighbors (excluding r itself), and then call topKNeighbors() to select the k closest neighbors. For each selected neighbor, we transform its distance into an edge weight using distanceToWeight(), and store that weight at position (r,\ \text{neighbor.Index}) in a new dense matrix. In the resulting directed matrix, each cell has outgoing edges to its k nearest neighbors.

getNeighborsRow() prepares the full candidate neighbor list for a single cell. Given one row of the distance matrix and the row index r, we iterate over all columns c, skip the self-distance (c = r), and append a Neighbor{Index: c, Distance: row[c]} entry for every other cell. This produces the complete set of potential neighbors for cell r, which we later filter down using *topKNeighbors()*.

topKNeighbors() selects the k nearest neighbors from a slice of Neighbor structs. We sort the slice in ascending order by Distance and return the first k entries. If fewer than k candidates are available, we simply sort and return all of them. In this way, we ensure that only the closest neighbors contribute edges to the KNN graph, keeping the graph sparse and preserving local structure.

distanceToWeight() converts a scalar distance into an edge weight using the transformation

$$w = \frac{1}{1+distance} \tag{14}$$

With this mapping, smaller distances produce larger weights, so nearby cells receive stronger similarity values. We apply *distanceToWeight()* to every selected neighbor pair inside *fillDirectedKNNWeights()*, ensuring the directed KNN matrix encodes similarities rather than raw distances.

symmetrizeWeightsUsing() turns the directed KNN weight matrix into an undirected graph that better reflects biological intuition. For each unordered pair of nodes (i, j) with $i < j$, it reads the directed weights $weight_{ij}$ and $weight_{ji}$: If both are zero, we skip the pair. Otherwise, we define the undirected weight as the maximum of the two directions and add symmetric edges ij with this weight.

Meanwhile, we accumulate all undirected weights into *totalWeight*. This symmetrization ensures that if cell i is strongly connected to cell j in either direction, we treat the relationship as mutual in the final KNN graph, while preserving sparsity and computational efficiency.

Community Identification: Leiden Algorithm

Using the KNN graph constructed in the previous step as a starting point, we now apply the Leiden algorithm to cluster cells of similar gene expression profiles into clusters, or communities. Here the terms community and cluster are used interchangeably to refer to a mutually exclusive partition of the graph. Since the Leiden algorithm builds upon and improves the Louvain algorithm, we begin by reviewing the fundamental principles of the Louvain method.

Louvain is a greedy clustering algorithm for detecting communities. It transforms the computational problem of identifying communities into one of optimizing the modularity gain function, a measure of community strength. It consists of two main phases: the local move nodes phase and the aggregation phase. In the first phase, nodes are iteratively moved into communities that result in the highest modularity gain. The process is repeated until the structure stabilizes and no further node moves result in an improved structure. Then, the nodes in each cluster are aggregated into a supernode, and the process repeats until the global structure converges.

The Leiden algorithm improves upon Louvain by ensuring that all communities are well connected and γ-separated. Well-connected communities are ensured through a Breadth-First Search (BFS) style evaluation of nodes in the local move nodes phase, implemented as *MoveNodesFast()* in our code. Separation is ensured by an additional refinement step, which examines clusters returned by the local move nodes phase and identifies sub-clusters of a size proportional to the resolution parameter γ.

Computational Problem

The modularity change of moving node i out of community D and into C is given by:

$$\Delta Q = Q(D \to i \to C) = Q(D \to i) + Q(i \to C)$$

Cluster density is the fraction of edge weights within nodes in the cluster over the total degree of nodes in the cluster. A high density indicates that the nodes are more tightly connected to each other than they are to nodes outside of the cluster. If this is the sole metric of cluster strength, the strongest community would trivially be the one containing all nodes.

Modularity Q is a measure of cluster strength, found by comparing the observed density of edges within a cluster compared to the density expected if the graph's edge weights were arranged at random. If the observed density is equivalent to what is expected by chance, then the cluster cannot truly be considered strong. Thus modularity of a single cluster *C* can be expressed in its simplified form as:

$$Q(C) = \frac{\sum_{in}}{2m} - \left(\frac{\sum_{tot}}{2m}\right)^2 \tag{16}$$

Where *2m* is the sum of all edge weights in an undirected graph, $\sum_{in}$ is the sum of edge weights between the nodes in cluster *C*, and $\sum_{out}$ is the total edge weight of all nodes in *C* (Figure 1).

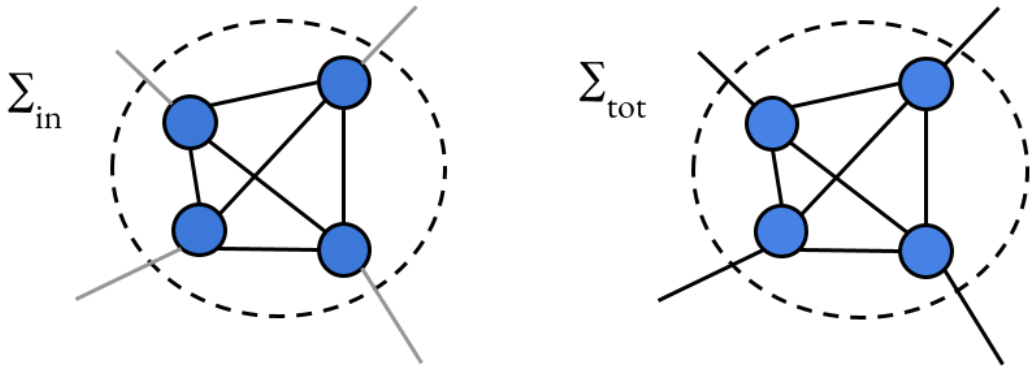


Figure 1. *Visualization of* $\sum_{in}$ *and* $\sum_{tot}$ *for a given cluster. Gray edges are not counted in* $\sum_{in}$

Following the convergence of the local move nodes phase, Leiden evaluates each cluster for further partition into γ-separated subclusters. First, it identifies a subset of nodes in the cluster that are well-connected.

Let R be the subset of nodes in cluster S that are sufficiently well connected to the rest of the cluster. Each node $v \in R$ must be a member of the subset S, and the sum of edge weights between v and all other nodes in the cluster excluding itself $E(v,S\vec{)}$ must be greater than or equal to some threshold based on the resolution parameter γ and the degrees of the node v and cluster S.

$$R = \{v \mid v \in S,\ E(v, S \setminus v) \geq \gamma \, \|v\| \cdot (\|S\| - \|v\|)\} \tag{17}$$

Each node $v \in R$ is then initialized into its own singleton community, forming a new partition $\mathcal{P} = \{\{v_i\} \mid v \in S\}$. For each cluster of the candidate nodes, a connectivity metric is computed. If the cluster meets the threshold described by the following equation, it is added to a list of candidate clusters that a well-connected node identified in the previous step might move to.

$$T \leftarrow \{\, C \mid C \in \mathcal{P},\ C \subseteq S,\ E(C, S \setminus C) \geq \gamma \, \|C\| \cdot (\|S\| - \|C\|) \,\} \tag{18}$$

Let T be a set of well-connected clusters. A well-connected cluster $C \in T$ must also be a member of the original partition $\mathcal{P}$, and it must be a subset of the set of nodes being evaluated for refinement S. If the sum of edge weights between nodes in C and nodes in $S \setminus C$ are greater than the threshold value, the cluster is added to the list of well-connected clusters.

Each singleton $v_i \in \mathcal{P}$ is iteratively evaluated in random order to determine if it can be merged with another singleton $v_{j,i \neq j}$ according to the modularity function ΔQ given above.

$$\Pr(C' = C) \sim \begin{cases} \exp\left(\frac{1}{\theta} \Delta Q(v \to C)\right), & \text{if } \Delta Q(v \to C) \geq 0,\ C \in \mathcal{P}, \\ 0, & \text{otherwise} \end{cases} \tag{19}$$

Unlike the local move nodes phase, any single-node move which results in $\Delta Q \geq 0$ is eligible. The final assignment is controlled by a randomness parameter θ. This process is repeated iteratively, such that a community in $\mathcal{P}$ might consist of more than one node at a certain time point, until no singleton communities are left.

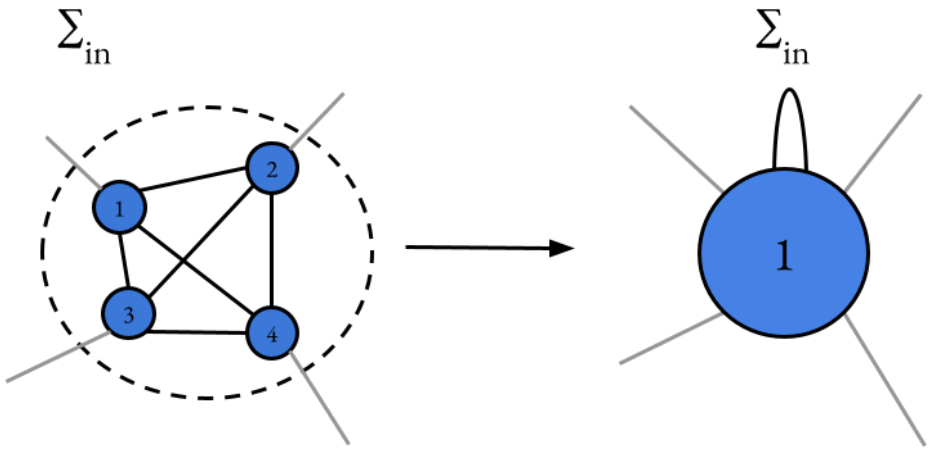


Figure 2. *Cluster Aggregation. The resulting supernode has a self loop with edge weight equal to* $\sum_{in}$

Following refinement, each subcluster is aggregated into a supernode, where the sum of edge weights within the cluster is represented by a self-loop (Figure 2).

High-Level Algorithmic Solution

We implement the Leiden algorithm on a KNN graph in the *Leiden()* function, which calls the main subroutines *InitSingletonPartition()*, *MoveNodesFast()*, *RefinePartition()*, *Refine()*, and *Aggregate()*. A global mapping of original node IDs to their current cluster labels is maintained throughout the function and updated at the end of each *Aggregate()* stage. A high level overview of each function is provided below.

InitSingletonPartition()

A **Graph* method that returns a slice of integers clusters. Each index in clusters is the ID of node *i*, and *clusters[i]* returns the cluster label of node *i*. This method is called on a deep copy of the input KNN graph, generated by the helper function *DeepCopy()*.

MoveNodesFast()

A helper method *NewQueue()* is called on the working graph, placing all nodes in the workingGraph within the BFS queue in random order by calling *RandomNodeOrder()* which implements Fisher-Yates swapping. The BFS object consists of an integer slice-based queue of node IDs and a boolean map marking which nodes are currently in the queue.

Helper methods *Enqueue()* and *Dequeue()* add and remove nodes from the queue. Upon enqueuing node *i*, the value of *inQueue* at key *i* is set to true. Similarly, each time node *i* is dequeued, the corresponding value in *inQueue* is set to false.

MoveNodesFast() takes as input the **BFS* object, along with the mapping of nodes to cluster IDs, and a resolution parameter. A node can only move into the communities of its neighboring nodes, so we call *FindCandidateClusters()*, which returns a list of the cluster labels that node *i*'s neighbors belong to.

We then call *FindBestCluster()* on the output of *FindCandidateClusters()*. For each of the candidate clusters, the function computes the modularity gain of moving node i into that cluster by calling the function *ModularityGain()*. *FindBestCluster()* identifies and returns the label of the cluster that maximizes modularity gain for moving node *i*.

ModularityGain() computes ΔQ of removing a given node from its existing community C and into a candidate cluster C'. Based on the equation for modularity given above, ΔQ is computed by the following formula:

$$\Delta Q = \Delta Q_{in} - \Delta Q_{out} \tag{20}$$

ΔQ_{in} is the modularity gain of moving a node into the target community C'

$$\Delta Q_{in} = \frac{k_{i,\text{in}(C')}}{m} - \gamma \left(\frac{K_{C'} k_i}{2m^2} + \frac{k_i^2}{4m^2} \right) \tag{21}$$

Similarly, ΔQ_{out} is the modularity gain of moving a node out of its current community C

$$\Delta Q_{out} = \frac{k_{i,in(C)}}{m} - \gamma \left(\frac{K_C k_i}{2m^2} - \frac{k_i^2}{4m^2} \right) \tag{22}$$

$k_{i,in(C)}$ is the $\sum_{in}$ of edge weights between the node i and the nodes in a given cluster C, γ is the resolution parameter, and m is the total edge weight. For an undirected graph, each edge weight is added twice.

If a node is moved into a different community, all of its neighbors which are not already in the queue are enqueued for further evaluation. This BFS style evaluation is the essence of Leiden and guarantees well-connected communities. Because every node whose connectivity might be disrupted by a neighbor's movement is forced back into the queue, we effectively eliminate the formation of disconnected communities.

When the queue eventually becomes empty, it implies that no further single-node move can yield additional modularity gain, indicating that the local structure has converged. We are now ready for the refinement stage of Leiden.

RefinePartition()

This function evaluates the output of *MoveNodesFast()*. A helper function *NodesByCluster()* creates a mapping of cluster labels to the subset of node IDs in that cluster. Then, each node in the subset is initialized as a singleton community. This is represented by a slice of integers *refinedPartition()* where the index is the node ID and the value of a given index is its cluster.

Then, *MergeNodesSubset()* walks through each singleton community in random order. It identifies well connected nodes and clusters that the current node might be merged with based on the threshold values. For each node in the singleton partition it is evaluating, *FindWellConnectedNodes()* returns a

list of nodes within the subset that are sufficiently well connected within its cluster according to the threshold parameter (Eqn. 17). Only well-connected nodes are considered as candidates for further refinement operations, as moving weakly connected nodes may disrupt the γ separation guarantee of the Leiden algorithm.

FindWellConnectedClusters() identifies candidate clusters that the well-connected node might move into. All clusters of the well-connected nodes are initially considered, but only clusters that meet a certain connectivity threshold as defined by Eqn. 18 are included in the final list of well-connected clusters.

ComputeMoveProbability() computes the modularity gain of a given well-connected node into one of the well-connected clusters. From modularity gain, the probability of the node moving into the cluster is computed according to Eqn. 19 for all candidate clusters identified in *FindWellConnectedClusters()*. The function only considers single-node moves resulting in a positive modularity gain; the probability of a node moving into a given cluster is 0 if modularity gain is negative.

Finally, *SampleCommunity()* selects the final cluster to which the node will be reassigned. It samples from the set of candidate clusters using the probabilities computed in the previous step.

MergeNodesSubset() is repeated iteratively until all nodes have been merged and no isolated nodes remain in a singleton community.

Aggregate()

At this stage, cluster labels may be non-sequential; for example, only clusters 38, 600, and 43 might remain after a few iterations. *Refine()* normalizes the labels so they are sequential, mapping these clusters to 1, 2, and 3. This is a cleanup step before aggregation and should not be confused with *RefinePartition()*, which performs the actual Leiden refinement.

Aggregate() collapses each cluster in a partition into a single node. Edges between the new nodes represent the total weight of all edges connecting the corresponding clusters in the original graph. The resulting graph has fewer nodes, with inter-cluster connections preserving the summed connectivity of the original graph.

ExportLeiden()

The results of Leiden clustering are visualized by labeling plots on the visual embeddings. The final mapping of node IDs to cluster labels are exported as a CSV using the *ExportLeiden()* function. R code then maps each node in the visual embedding to a unique cluster color.

Results

To test our scRNA-seq analysis pipeline, we applied it to a real dataset derived from a mouse ischemic stroke model with astrocyte-specific deletion of the sodium bicarbonate transporter NBCe1. Using 10X Cell Ranger, we performed sequencing alignment and obtained our raw count matrix, which contains 40,148 cells, including astrocytes and other brain cell populations collected from a wild-type mouse following traumatic brain injury.

Because it is computationally demanding for a scratch implementation of the pipeline, we randomly subsampled 750 cells, which is the maximum number of cells limited by RAM, from the full dataset without replacement, retaining all genes for each selected cell. This helps us maintain the distribution of cells and expression variability while reducing memory usage and runtime to manageable levels.

While the original study focused on the role of astrocytic NBCe1 in ischemic injury and downstream metabolic and inflammatory signaling, we use this dataset only as a test case for validating the computational behavior of our pipeline. Because of this, the exact choice of dataset is not critical, provided the subsample remains sufficiently complex for us to test all stages of the pipeline.

Comparison of Leiden clustering between Go and Seurat pipelines.
To validate the correctness and behavior of our Go-based implementation, we compared Leiden clustering results from our pipeline with those generated by Seurat on the same PCA-reduced dataset. In both cases, clustering was performed on the top 30 PCs using identical parameters.

Clusters produced by the Go pipeline appear less tightly packed than those from Seurat, resulting in more diffused cluster boundaries in the embedding (Figure 3). This difference likely reflects implementation choices, as we only expose a limited set of tunable parameters and use slightly simplified implementations of normalization and graph construction, prioritizing clarity and correctness over visual refinement. In contrast, Seurat uses multiple highly optimized routines which tend to produce more compact clusters.

Despite these differences, both approaches recover similar coarse-grained community structure, with major cell groupings appearing consistently across embeddings. In addition, some clusters show clear correspondence between the two pipelines. For example, Seurat cluster 7 aligns closely with Go cluster 11, and Seurat cluster 5 with Go cluster 9, as shown by their overlapping locations in the embedding space (Figure 3). These pairs also have a similar local structure, especially for 7 and 11, which indicates that they represent the same underlying groups of cells. Thus, based on the agreement between the clustering structure produced by Seurat and our Go pipeline, we believe that our implementation is able to faithfully reproduce key aspects of Leiden-based community detection.

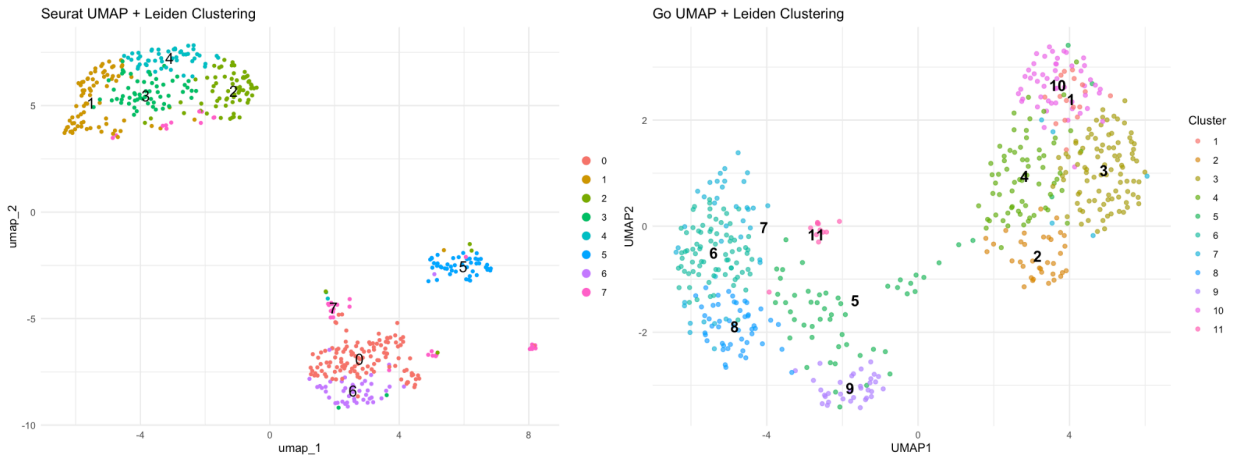


Figure 3 - Comparison of the Leiden clustering algorithm between the Go pipeline and Seurat. Clustering was performed on a PCA-reduced dataset that has 30 principle dimensions. Left: visualization using Seurat's Leiden clustering and Seurat's UMAP. Right: visualization using Go Leiden clustering and Seurat's UMAP. The parameter used in clustering for both plots is: k=10 and resolution=1.

Comparison of Log Normalization and Analytical Pearson Residuals

To see how implementation and preprocessing choices can affect downstream results, we compared log-normalization and Pearson residual normalization across t-SNE and UMAP embeddings. Our result demonstrates that, in t-SNE, Pearson residuals appear to produce less distinct clusters, with cluster 2 position within the region occupied by Cluster 1 (Figure 4a). In contrast, the data normalized using log normalization (Figure 4b) yields clusters that are more clearly separated. However, because t-SNE prioritizes preservation of local neighborhood structure rather than inter-cluster distance, the cluster overlap does not necessarily reflect true biological similarity or normalization performance.

In UMAP, both Pearson and log normalization produce well-defined, distinct clusters, indicating broadly similar preservation of cellular structure. Notably, Pearson normalization produces 6 clusters (Figure 4c), while log normalization (Figure 4d) only produces 4. Additionally, Pearson seems to result in differently shaped islands. Each island is more closely packed together. Because UMAP better preserves both local and some global structure, these differences may reflect improved separation of biologically meaningful variation, although we cannot draw definitive conclusions of normalization based on this visual embedding alone. Together, these results suggest that a more complex model does not necessarily lead to improved biological signal preservation, underscoring the importance of providing researchers with tools to systematically compare normalization methods on their own datasets. When simpler approaches are shown to perform comparably, researchers can confidently adopt computationally efficient methods where appropriate.

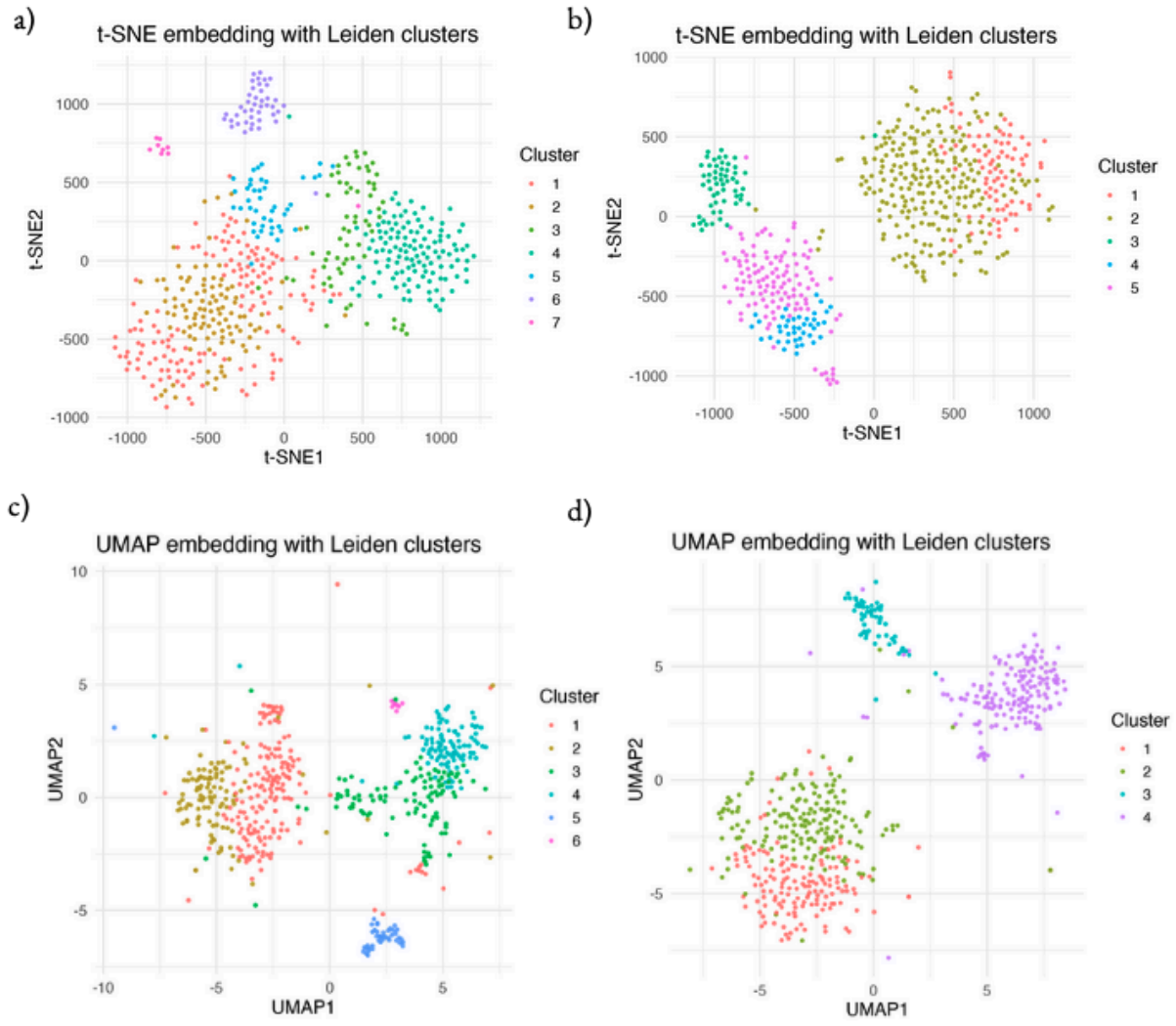


Figure 4 - Visualization of clustering results using different combinations of normalization method and embedding algorithm. The data was preprocessed and reduced to 30 dimensions using PCA. All four plots share the same Leiden clustering parameter (k=15 and resolution = 0.3). a) Pearson residual and t-SNE(perplexity=25, learning rate=100, iteration=1000). b) Log-Normalization and t-SNE (perplexity=25, learning rate=100, iteration=1000). c) Pearson residual and UMAP (n_neighbor=30, learning rate=0.2). d) Log-Normalization and UMAP (n_neighbor=30, learning rate=0.2).

Discussion

In this project, we built a scRNA-seq analysis pipeline in Go by reimplementing the core computational steps used in widely adopted single-cell workflows, which includes quality control, log normalization, pearson residuals, PCA dimensionality reduction, KNN graph generation, Leiden community detection, and non-linear embedding methods like t-SNE and UMAP for visualization. The main objective of this project was to understand and reproduce the computational steps that

convert a sparse UMI count data into an human-interpretable low-dimensional structure, and be able to have graph-based clusters for easier interpretation.

A central outcome of our project is that we are able to recover a clustering structure that is highly comparable to Seurat under matched parameters. When Leiden clustering was performed on the same PCA-reduced data with identical parameters, the major groupings of cells were consistent across the two implementations (Figure 3). Specific cluster correspondences such as Seurat cluster 7 aligning with Go cluster 11 shows that our implementation captures similar neighborhood structure and community organization rather than arbitrary partitions. This validates that our implementation can actually reproduce key aspects of the analysis pipeline in standard single-cell analysis tools.

At the same time, our results also illustrate how normalization interacts with downstream steps, especially when paired with non-linear embeddings. While log-normalization and Pearson residuals produced broadly similar large-scale clusters, the number and visual separation of clusters varied across t-SNE and UMAP (Figure 4). This shows that preprocessing decisions during normalization can change downstream structures. The Seurat R package provides log normalization as its default method, and it is one of the most simple normalization methods for scRNA sequencing. Traditionally, a Poisson model is used to model scRNA-seq data. Gene expression can be modeled as a rare event, since a given gene is not expressed in many cells. However, the authors of the Seurat package argue that scRNA-seq data tends to exhibit substantial overdispersion[15]. As a result, they developed a normalization method using a negative binomial distribution. In contrast, others argue that a negative binomial model is overspecified[14]. They show that analytic Pearson residuals derived from a simpler Poisson model outperform other normalization methods. Together, these suggest that a more complex model does not necessarily lead to improved biological signal preservation, underscoring the importance of providing researchers with tools to systematically compare normalization methods on their own datasets. When simpler approaches are shown to perform comparably, researchers can confidently adopt computationally efficient methods where appropriate.

The project also has several limitations. First, due to computational and memory limitations, we were only able to analyze a randomly subsampled set of cells rather than the full dataset, which limits our ability to fully capture rare populations and fine grained structure. In addition, the evaluation of clustering performance is largely qualitative against an established tool rather than on benchmarking metrics. Finally, while we planned the project with concurrent and parallel computing in mind, we did not end up implementing it. While the project runs fast on a small subsample, we are not sure about the performance in a full size dataset. Nonetheless, this work highlights both the robustness of graph based clustering methods and the value of understanding how computational design decisions influence data interpretation.

Project Contributions (ordered by the amount of contribution)

	Code	Presentation	Final Essay
Introduction	n/a	VH, QK, YZ	QK, AJ
Quality Control	QK	QK	QK
Pearson Residuals	VH	AJ	QK
Log Normalization	VH, QK	AJ	QK
PCA (used package)	QK, AJ	QK	QK
UMAP	AJ	AJ	AJ
t-SNE (used package)	AJ, YZ	AJ	AJ
KNN Graph	VH, QK	VH	YZ, VH
Leiden Clustering	VH, QK, AJ	VH	VH
Unit Tests	YZ, AJ, QK	n/a	n/a
R-Shiny App	AJ	n/a	n/a
Results	n/a	n/a	AJ, QK, VH
Discussion	n/a	n/a	QK, VH

References

1. Macosko, E. Z., Basu, A., Satija, R., Nemesh, J., Shekhar, K., Goldman, M., ... & McCarroll, S. A. (2015). Highly parallel genome-wide expression profiling of individual cells using nanoliter droplets. *Cell, 161*(5), 1202–1214. https://doi.org/10.1016/j.cell.2015.05.002

2. Koch, C. M., Chiu, S. F., Akbarpour, M., Bharat, A., Ridge, K. M., Bartom, E. T., & Winter, D. R. (2018). A beginner's guide to analysis of RNA sequencing data. *American Journal of Respiratory Cell and Molecular Biology, 59*(2), 145–157. https://doi.org/10.1165/rcmb.2017-0430TR

3. Danaugrs. (n.d.). *go-tsne: t-Distributed Stochastic Neighbor Embedding (t-SNE) in Go* [Computer software]. GitHub. https://github.com/danaugrs/go-tsne

4. Whitworth, S. J. (n.d.). *Golearn: PCA package* [Computer software]. GitHub. https://pkg.go.dev/github.com/sjwhitworth/Golearn/pca

5. NCBI GEO. (n.d.). *Expression data from Drop-seq of mouse retina cells (GSM1626794)* [Data set]. National Center for Biotechnology Information. https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSM1626794

6. Traag, V.A., Waltman, L. & van Eck, N.J. From Louvain to Leiden: Guaranteeing well-connected communities. *Sci Rep* 9, 5233 (2019). https://doi.org/10.1038/s41598-019-41695-z

7. Heumos, L., Schaar, A., & single-cell best practices consortium. (2023). *Clustering*. *Single-cell best practices*. Retrieved December 12, 2025, from https://www.sc-best-practices.org/cellular_structure/clustering.html

8. McInnes, L., Healy, J., & Melville, J. (2020). *UMAP: Uniform manifold approximation and projection for dimension reduction* (arXiv:1802.03426v3). arXiv. https://doi.org/10.48550/arXiv.1802.03426

9. van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research, 9*, 2579–2605. https://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf

10. Awan, A. A. (2024, December 9). *Introduction to t-SNE*. DataCamp. https://www.datacamp.com/tutorial/introduction-t-sne

11. Thevapalan, A. (2025, November 4). Understanding UMAP: A comprehensive guide to dimensionality reduction. DataCamp. Retrieved December 12, 2025, from https://www.datacamp.com/tutorial/understanding-umap-guide-to-dimensionality-reduction

12. Amezquita, R., Lun, A., Hicks, S., & Gottardo, R. (2021, May 21). *Chapter 5: Clustering, redux*. In *Advanced Single-Cell Analysis with Bioconductor* (Bioconductor 3.13). Bioconductor. https://bioconductor.org/books/3.13/OSCA.advanced/clustering-redux.html

13. Hvitfeldt, E., & Bodwin, K. (n.d.). *Measures silhouette between clusters — silhouette (tidyclust 0.2.4)*. tidyclust (tidymodels). Retrieved December 12, 2025, from https://tidyclust.tidymodels.org/reference/silhouette.html

14. Lause, J., Berens, P. & Kobak, D. Analytic Pearson residuals for normalization of single-cell RNA-seq UMI data. Genome Biol 22, 258 (2021). https://doi.org/10.1186/s13059-021-02451-7

15. Hafemeister, C., Satija, R. Normalization and variance stabilization of single-cell RNA-seq data using regularized negative binomial regression. Genome Biol 20, 296 (2019). https://doi.org/10.1186/s13059-019-1874-1